

PARALLAX ENGINEERING

Backend Implementation and Integration Plan

A four-person delivery blueprint for independent development, safe integration, transaction correctness, security, verification, and production launch.

01	02	03	04
Database foundation	Identity and access	Fest operations	Progression and platform

SOURCE BASELINE

Uploaded PARALLAX backend requirements and the recovered backend task-division contract.

Version 1.0 | 31 July 2026

Document control

This plan is the execution baseline for splitting the PARALLAX backend across four engineers and merging it into one production-ready system.

Field	Value
Document	PARALLAX Backend Implementation and Integration Plan
Purpose	Define ownership, interfaces, transactions, tests, integration order, and release gates.
Source	Uploaded PARALLAX backend requirements plus the source-derived repository and schema inventory recovered from the referenced conversation.
Baseline	13 repository areas, 58 operations, 27 tables, 2 database roles, shared DataError contract.
Audience	Four backend implementers, frontend integrators, reviewers, release owner, and faculty/project stakeholders.
Change rule	Contract or schema changes require an ADR, impact note, migration, and approval from affected owners.
Status	Ready for team assignment and sprint planning.

How to use this plan

- **Start with Sections 1-3:** freeze the architecture, interface, error, and ownership boundaries before implementation begins.
- **Each engineer owns one lane:** implementation, migrations for owned tables, tests, operational notes, and review of changes that touch that lane.
- **Use the transaction catalog:** do not improvise lock order or move privileged writes to the normal database connection.
- **Integrate by sequence:** merge contract and database foundations before mutation-heavy features.
- **Release only through gates:** all security, concurrency, migration, restore, and observability gates must be evidenced.

Decision authority

The repository interface and database schema are shared contracts. A developer may refactor inside an owned module, but may not silently change a cross-team type, error code, table column, unique constraint, or transaction invariant.

Contents

Document control	2
How to use this plan	2
Contents	3
1. Executive implementation plan	6
Four-person division	6
Independence model	6
Primary integration risks	6
2. Shared architecture and contracts	7
Required request path	7
Repository contract rules	7
Recommended source layout	7
Database connection policy	8
MySQL session settings	8
3. Source requirements inventory	9
Table inventory and stewardship	9
Required views and indexes	9
Stable DataError codes	10
Error implementation rules	10
Suggested transport mapping	10
4. Person 1 - Database foundation and repository infrastructure	11
Owned components	11
Component-by-component operations	11
Migration series	11
Seed requirements	11
Deliverables and completion criteria	12
Dependencies and handoffs	12
5. Person 2 - Authentication, authorization, profiles, and characters	13
Owned repository operations	13
Authentication components and flows	13
Authorization model	13
Profile and character field policy	14
Security and test ownership	14
Completion criteria	14
6. Person 3 - Events, registrations, teams, attendance, and QR check-in	15
Owned repository operations	15

Event reads	15
Registration transaction	15
Cancellation and waitlist promotion	15
Team operations	16
QR token and check-in	16
Concurrency and test ownership	16
Completion criteria	16
7. Person 4 - XP, achievements, payments, announcements, and storage	17
Owned repository operations	17
XP ledger and leaderboard	17
Achievement grants	17
Payment receipt lifecycle	17
Storage workflow	17
Announcements	18
Test ownership and completion criteria	18
8. File ownership and merge boundaries	19
Migration ownership protocol	19
Shared file protocol	19
9. Branch and pull request strategy	20
Branch model	20
Commit and pull request rules	20
Pull request template	20
Definition of done for a pull request	20
10. Transaction and consistency catalog	21
Deadlock and retry policy	21
Outbox / post-commit side effects	21
Invariants to assert	21
Reconciliation and consistency controls	22
Consistency incident response	22
11. Security and privacy rules	23
Identity and session	23
Authorization and data access	23
Input, abuse, and content controls	23
Secrets and environment	23
Audit logging	23
Retention and privacy	23
Required security evidence	24
Security incident readiness	24

12. Test ownership and quality strategy	25
Minimum test layers	25
Required fixtures	25
Coverage and evidence	25
Performance targets to agree before load test	26
Load-test design	26
CI gate order	26
Test-data lifecycle	26
13. Integration sequence and release train	27
Wave 0 - contract freeze	27
Wave 1 - database foundation	27
Wave 2 - identity plus read paths	27
Wave 3 - core participant transactions	27
Wave 4 - privileged operational flows	27
Wave 5 - hardening and release	27
Integration ownership	28
Wave exit criteria	28
Cross-lane integration smoke suite	28
14. Go-live gates and operations	29
Deployment sequence	29
Rollback / forward-fix triggers	29
Operational dashboards	30
Launch-day command roles	30
Incident severity during launch	30
15. Team operating model	31
RACI summary	31
Cadence	31
Decision records required	31
Escalation rules	31
Appendix A - Operation acceptance matrix	32
Appendix B - Schema ownership matrix	34
Appendix C - Environment contract	35
Environment promotion checklist	35
Appendix D - Review checklists	36
Transaction review	36
Authorization review	36
Migration review	36
Release review	36

1 Executive implementation plan

1. Executive implementation plan

PARALLAX should be divided by backend domain, not by arbitrary file count. The recommended split minimizes simultaneous edits, keeps privileged workflows with clear owners, and preserves one repository contract between the frontend and concrete MySQL implementations.

4	13	58	27	2
engineering lanes	repository areas	contract operations	database tables	database roles

Four-person division

Owner	Mission	Repository areas	Primary outcome
Person 1 - Foundation	Database, schema, connections, migrations, shared repository infrastructure.	reference plus shared database infrastructure	A reproducible, least-privilege data platform that every other lane can build on.
Person 2 - Identity	Authentication, authorization, session security, roles, profiles, character identity.	auth, profiles, characters	A trustworthy caller identity and server-side permission layer.
Person 3 - Fest operations	Events, registrations, teams, attendance, QR check-in.	events, registrations, teams, attendance	Concurrency-safe event operations with no overbooking or token replay.
Person 4 - Progression	XP, achievements, payments, announcements, storage.	progression, achievements, payments, announcements, storage	Auditable privileged workflows and resilient platform services.

Independence model

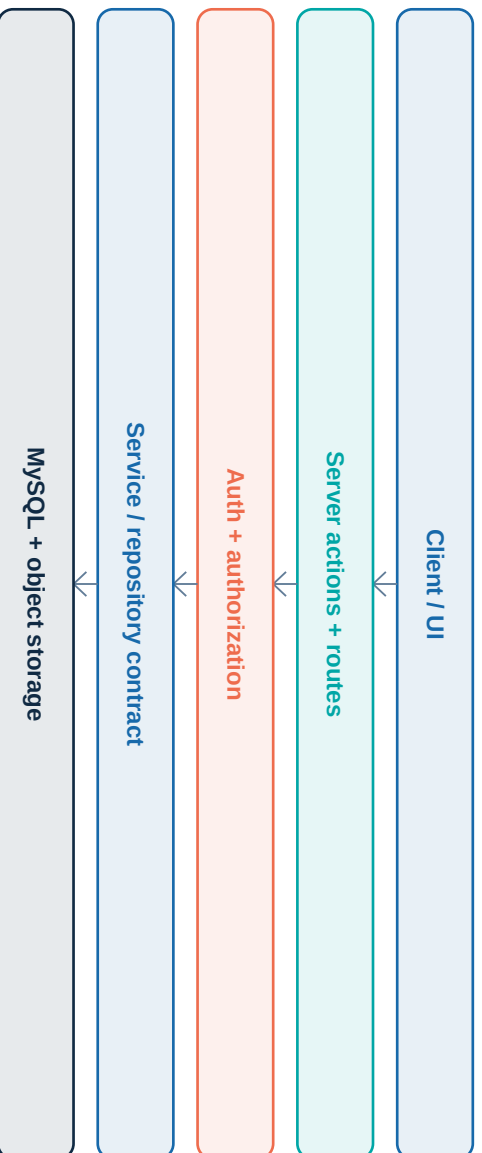
- Each lane owns its server modules, schemas for request validation, concrete repository implementation, service rules, tests, and operational documentation.
- Cross-lane calls use typed interfaces or service commands. No lane imports another lane's concrete SQL implementation.
- Shared tables are changed through migration proposals reviewed by Person 1 and every affected domain owner.
- Privileged operations use the writer connection and always create an audit record in the same transaction where practical.
- Frontend code calls server actions or route handlers; it never receives credentials or imports Drizzle, mysql2, storage secrets, or privileged helpers.

Primary integration risks

Risk	Consequence	Control
Capacity race	Overbooking or inconsistent waitlist.	Lock event row, use locking count, one transaction, unique constraint.
Authorization drift	Organizer reads or mutates another event.	DB-backed permission check on every protected mutation.
Privilege leakage	Normal app credential awards XP or approves payment.	Separate writer user, grant tests, server-only module boundary.
Interface drift	Frontend and lanes disagree on method shape or errors.	Contract freeze, generated types, contract tests, CODEOWNERS.
Migration conflict	Branches create incompatible or reordered schema history.	Person 1 allocates migration numbers and serializes shared-schema merges.
Token replay	A copied QR code checks in twice.	Short-lived signed token, jti redemption table, unique constraint, atomic check-in.

2 Shared architecture and contracts

All lanes implement behind one server-side boundary. The interface is the integration seam; the database is an implementation detail.



Required request path

Client -> server action or route -> authentication -> authorization -> Zod validation -> service -> repository -> database/storage -> domain result or DataError.

Repository contract rules

- Inputs and outputs are serializable domain types; dates cross boundaries as UTC ISO strings.
- Repository methods do not expose Drizzle result rows, driver error objects, secrets, or internal table names.
- Expected business failures use stable DataError codes. Unexpected infrastructure failures are logged with a correlation ID and return a safe public message.
- Mutations receive actor identity from the server session, never from a client-supplied userid.
- Pagination is cursor-based for unbounded lists; ordering is explicit and stable.
- Idempotency keys are required for externally retried payment, storage completion, check-in, and privileged award operations.

Recommended source layout

```
src/  
  backend/auth/  
  backend/db/{app-db,writer-db,transaction,schema,migrations}/  
  backend/repositories/{repository,errors,mysql}/  
  backend/services/{events,registrations,teams,attendance,progression,payments}/  
  backend/security/{authorization,rate-limit,tokens,audit}/  
  backend/storage/  
  backend/validation/  
  app/actions/ and app/api/  
  middleware.ts
```

Database connection policy

Connection	Used for	Must not do	Controls
Application DATABASE_URL	Public reads, profile updates, event reads, registrations, team membership, user-scoped reads.	Award XP, grant roles, record attendance, verify payment, change audit history.	TLS, UTC, READ COMMITTED, bounded pool, statement timeout, least grants.
Writer WRITER_DATABASE_URL	XP awards, role grants, attendance, payment verification, audit-sensitive writes.	Client-side use or broad ad hoc queries.	Server-only import, smaller pool, 2FA for human access, audit, strict grants.

MySQL session settings

- **timezone = UTC** so DATETIME(3) fields and tokens behave consistently.
- **transaction isolation = READ COMMITTED** so capacity and membership counts see committed changes while explicit locks enforce correctness.
- **bigNumberStrings = true** so BIGINT values do not lose precision in JavaScript.
- **dateStrings = true** so the domain layer controls ISO conversion.
- Set a bounded lock wait timeout and apply a maximum of three deadlock retries only for MySQL 1213 and 1205.

Non-negotiable invariant

The normal application credential must fail when attempting to modify xp_ledger, attendance, user_roles, payment verification fields, or protected audit data.

3 Source requirements inventory

3. Source requirements inventory

This inventory converts the source document into implementation-sized units. The method list below is the recommended canonical contract for the four-lane build and totals 58 operations across 13 areas.

Area	Count	Operations	Owner
auth	7	signUp, signIn, signInWithProvider, signOut, getSession, onAuthStateChanged, grantRole	P2
profiles	2	get, update	P2
reference	5	listColleges, listDepartments, listCategories, listLevels, listSponsors	P1
characters	3	getForUser, create, update	P2
events	5	list, getBySlug, getById, stats, schedule	P3
registrations	5	listForUser, listForEvent, get, register, cancel	P3
teams	7	listForUser, getById, getByJoinCode, members, create, join, leave	P3
attendance	3	listForUser, listForEvent, checkIn	P3
progression	5	getSummary, listLedger, leaderboard, awardXp, recalculateTotal	P4
achievements	4	list, listForUser, grant, markSeen	P4
payments	5	listForUser, listForEvent, get, submitReceipt, verifyReceipt	P4
announcements	3	list, create, archive	P4
storage	4	createUploadUrl, completeUpload, createDownloadUrl, deleteObject	P4

Table inventory and stewardship

Group	Tables	Steward
Auth	users, accounts, sessions, verification_tokens	P2; schema merge by P1
Identity	profiles, user_roles, colleges, departments	P2 for behavior; P1 for reference data
Progression	characters, levels, xp_ledger, achievements, user_achievements	P4; character identity coordinated with P2
Events	event_categories, events, event_organizers, schedule_slots, announcements	P3; announcements behavior by P4
Participation	teams, team_members, registrations, attendance, checkin_token_redemptions	P3
Platform	payment_receipts, audit_log, sponsors, certificates	P4; audit infrastructure shared

Required views and indexes

- **leaderboard view:** total XP descending with created_at ascending as a stable tie-breaker.
- **event_stats view:** confirmed, waitlisted, checked-in, and remaining-seat counts.
- Indexes: characters(total_xp DESC, created_at ASC); events(status, starts_at); registrations(event_id, status); registrations(user_id, registered_at DESC); attendance(event_id, checked_in_at DESC).
- Indexes: xp_ledger(user_id, created_at DESC); user_achievements(user_id, seen_at); announcements(published_at DESC); payment_receipts(status, submitted_at).

Stable DataError codes

Code	Meaning / handling
NOT_AUTHENTICATED	No valid server session. Return 401 or redirect to sign-in.
INVALID_CREDENTIALS	Generic credential failure; do not reveal whether an account exists.
EMAIL_TAKEN	Normalized email already exists.
PLAYER_NAME_TAKEN	Character/player name uniqueness conflict.
NOT_FOUND	Resource missing or intentionally hidden from an unauthorized caller.
ALREADY_REGISTERED	Duplicate event registration or conflicting membership.
EVENT_FULL	No capacity and waitlisting is disabled.
REGISTRATION_CLOSED	Registration window or state disallows change.
TEAM_FULL	Locked membership count reached maximum.
INVALID_JOIN_CODE	Unknown, expired, or rotated team join code.
STORAGE_UNAVAILABLE	Storage or signed URL service is unavailable.
VALIDATION_FAILED	Input failed server-side schema or domain validation.
RECEIPT_ALREADY_SUBMITTED	An active receipt already exists for the registration/payment.
PAYMENT_NOT_VERIFIED	Operation requires a verified payment.

Error implementation rules

- Construct DataError only through shared helpers so code, safe message, HTTP status, retryability, and correlation ID stay consistent.
- Log the internal cause server-side, but return only the stable code and a user-safe message.
- Duplicate-key errors are mapped using the violated named constraint; do not map every duplicate to the same business code.
- Authorization failures never include the caller's role list, expected role, SQL, or target existence.
- Infrastructure unavailability may be retryable; validation, permission, and uniqueness failures are not.

Suggested transport mapping

HTTP	Error families	Client behavior
400	VALIDATION_FAILED, invalid state/input	Show field or action guidance; do not retry unchanged request.
401	NOT_AUTHENTICATED, INVALID_CREDENTIALS	Prompt sign-in or credential correction.
403/404	scope/role denial or intentionally hidden target	Do not reveal protected target details.
409	EMAIL_TAKEN, PLAYER_NAME_TAKEN, ALREADY_REGISTERED, TEAM_FULL, receipt conflict	Refresh relevant state and offer a safe alternate action.
422	REGISTRATION_CLOSED, EVENT_FULL, PAYMENT_NOT_VERIFIED	Explain domain prerequisite without exposing internals.
503	STORAGE_UNAVAILABLE or controlled dependency outage	Preserve form state and offer bounded retry.

4

Person 1 - Database foundation

4. Person 1 - Database foundation and repository infrastructure

Mission

Provide a reproducible, typed, least-privilege database platform so the other three lanes can work without touching connection internals or inventing migrations.

Owned components

- Environment contract: .env.example, validation at process start, production/preproduction separation, secret naming, and server-only import boundaries.
- Application and writer connection pools, TLS verification, UTC/session settings, timeouts, pool sizing, and graceful shutdown.
- Central Drizzle schema for all 27 tables; exact foreign-key type compatibility; naming and timestamp conventions.
- Migration ordering, rollback notes, schema-check automation, deterministic seed data, and preproduction promotion.
- Shared utilities: withTransaction, withDeadlockRetry, mapDatabaseError, generateUuidV7, toDomainObject, and requireRecord.
- Reference repository, leaderboard and event_stats views, required indexes, and database grant tests.

Component-by-component operations

Component	Operation	Implementation responsibility	Acceptance evidence
Environment	loadConfig	Validate required variables; reject placeholders; never expose server secrets to client bundles.	Startup test fails clearly for each missing or malformed variable.
Connections	getAppDb / getWriterDb	Create bounded pools with TLS, UTC, READ COMMITTED, bigNumberStrings, dateStrings.	Connection health and grant tests pass in CI/preproduction.
Transactions	withTransaction	Begin, commit, rollback, preserve domain error, attach correlation context.	Forced failure leaves no partial rows.
Resilience	withDeadlockRetry	Retry only 1213/1205 with 50/100 ms backoff and maximum three attempts.	Deterministic unit test proves bounds and non-retriable behavior.
Errors	mapDatabaseError	Map duplicate, FK, not-found, permission, lock, and unavailable errors to safe DataError.	No driver message reaches API responses.
Reference	5 read methods	List seeded colleges, departments, categories, levels, and sponsors with stable ordering.	Contract tests and seeded snapshot pass.
Schema	27 tables	Define keys, enums/status constraints, foreign keys, timestamps, uniqueness, and sensitive-table separation.	Fresh database migrates from zero and schema check passes.
Views	leaderboard/event_stats	Create deterministic views and expose typed read models.	View results verified against fixture data.

Migration series

```
0001_auth_tables.sql
0002_reference_tables.sql
0003_event_tables.sql
0004_registration_and_team_tables.sql
0005_progression_tables.sql
0006_payment_storage_audit.sql
0007_views_indexes_and_grants.sql
```

Seed requirements

- Colleges and departments used by eligibility/profile forms.
- Seven event categories and seven progression levels.

- Approximately fifteen achievements with deterministic keys and thresholds.
- Sponsors and optional development fixtures for users, events, teams, receipts, and attendance.
- Seed logic is idempotent: rerunning it updates or skips stable keys and never duplicates rows.

Deliverables and completion criteria

Deliverable	Done when
db schema and migrations	A blank database reaches the current schema with no manual SQL.
connections and transactions	Both roles connect; the app role is denied protected writes.
seed and views	Seed reruns cleanly; view fixtures produce stable results.
shared helpers	All lanes can transact, retry, map errors, and generate IDs through one implementation.
schema CI	Migration, drift, index, and grant checks run on every pull request.

Dependencies and handoffs

- Publishes typed schema and transaction helpers before mutation work begins.
- Reviews migrations from Persons 2-4 within one working day.
- Coordinates any change to users.id, status enums, unique constraints, and audit columns.
- Does not implement domain authorization or business eligibility rules owned by other lanes.

5 Person 2 - Identity and access

5. Person 2 - Authentication, authorization, profiles, and characters

Mission

Establish who the caller is, which roles and event scopes they hold, and which identity/profile fields they may read or change.

Owned repository operations

Area	Operations	Notes
auth	signUp; signIn; signInWithProvider; signOut; getSession; onAuthStateChange; grantRole	Auth.js, credentials/OAuth, secure sessions, privileged role grant.
profiles	get; update	Field-level exposure and self-update rules.
characters	getForUser; create; update	Player-name uniqueness, safe cosmetic/profile fields, progression fields read-only.

Authentication components and flows

Sign-up

Validate -> normalize email -> check conflicts -> hash password with Argon2id or bcrypt -> create user/profile/default role in one transaction -> issue verification token -> send email -> return safe result.

- Use CHAR(36) ASCII user IDs consistently across Auth.js and all foreign keys.
- Do not roll back the account if email transport fails after a committed account transaction; persist a resendable verification state and return a controlled message.
- Rate-limit by normalized email and source network using privacy-aware keys.

Sign-in and session

Validate credentials -> fetch user -> verify hash -> check account status -> create secure session -> set httpOnly, Secure, SameSite=Lax/Strict cookie. Unknown email and wrong password both return INVALID_CREDENTIALS.

OAuth

Configure approved redirect URIs, state/nonce checks, account-linking rules, and first-login profile creation. Provider-supplied roles or client metadata are never authoritative.

Email verification and password reset

Verification and reset tokens are random, one-time, hashed at rest, scoped, and expire. Password reset returns a generic public response, updates the hash, invalidates the token, and revokes existing sessions.

Authorization model

Helper	Decision
assertAuthenticated	Valid server session; returns actor ID.
assertAdmin	Current DB role contains admin.
assertOrganizer	Current DB role contains organizer.
assertEventOrganizer	Actor is assigned to the exact event or is admin.
assertScannerPermission	Actor may scan for the exact event and current operational window.
assertPaymentReviewer	Actor has payment_reviewer/admin role and required 2FA assurance.

- Read roles from the database during protected mutations; do not trust a client cookie, localStorage, profile field, or stale page data.
- Route middleware improves navigation safety but does not replace authorization inside server actions or route handlers.
- Event organizers are event-scoped. Organizer A cannot list, scan, announce for, or modify Organizer B's event.
- Require 2FA for admins, payment reviewers, and other privileged operational users.

Profile and character field policy

Field class	Examples	Rule
Self-editable	name, phone, college, department, year, permitted public bio/avatar	Validate and update only caller-owned record.
Server-managed	email verification, roles, XP, level, attendance, payment status, admin flags	Reject if supplied by client.
Public	display/player name, selected public character fields	Expose through explicit public DTO only.
Operational	contact fields needed by organizers	Expose only for an authorized event and only the minimum required.

Security and test ownership

- Own tests for anonymous, unverified, player, organizer, scanner, payment reviewer, and admin access.
- Own session fixation, logout invalidation, reset-token replay, OAuth callback validation, account enumeration, and role escalation tests.
- Own field-level profile/character tests including PLAYER_NAME_TAKEN and protected-field rejection.
- Record audit entries for role grants, security-setting changes, and sensitive profile operations.

Completion criteria

- Browser-side SHA-256 or client-side password verification is absent.
- All protected mutations reread permissions server-side.
- Roles and sensitive profile fields cannot be modified by the application client.
- Organizer cross-event access fails in unit, integration, and API tests.
- Session cookies and token lifecycles meet the security rules in Section 11.

6 Person 3 - Fest operations

6. Person 3 - Events, registrations, teams, attendance, and QR check-in

Mission

Deliver the main fest workflows with explicit locks, stable lock order, database constraints, and server-side event-scoped authorization.

Owned repository operations

Area	Operations
events	list; getBySlug; getById; stats; schedule
registrations	listForUser; listForEvent; get; register; cancel
teams	listForUser; getById; getByJoinCode; members; create; join; leave
attendance	listForUser; listForEvent; checkIn

Event reads

- Public event list supports status, category, date, eligibility, availability, and individual/team filters; it is cacheable with explicit invalidation.
- getBySlug/getById returns public details, timing, venue, capacity, rules, team limits, schedule, and user-specific state only when a session exists.
- stats uses event_stats; detailed participant information requires assertEventOrganizer.
- schedule orders by starts_at and a deterministic venue/sequence tie-breaker.

Registration transaction

```
BEGIN
1. SELECT event FOR UPDATE
2. Validate state, time window, eligibility, team/payment/email requirements
3. Lock/read confirmed registration count
4. Insert CONFIRMED if capacity exists; otherwise WAITLISTED when enabled
5. Enforce UNIQUE(event_id, user_id)
6. Enqueue/record eligible progression side effect through the agreed integration seam
COMMIT
```

Lock rule

Registration and cancellation both lock the event before registrations. No operation may reverse this order.

Cancellation and waitlist promotion

Lock event -> lock caller registration -> mark cancelled -> lock earliest waitlisted row ordered by registered_at/id -> promote if eligibility still holds -> apply XP/payment policy -> commit. Promotion must be deterministic and auditable.

Team operations

Operation	Transaction and rules
create	Validate team event and eligibility; create random unique join code; add creator as leader in one transaction.
join	Resolve code; lock team; validate event/user; locking membership count; reject TEAM_FULL; insert membership; commit.
leave	Lock team then memberships; enforce deadline; transfer leadership or dissolve; revalidate team registration; commit.
join code	High entropy, non-sequential, unique, rate-limited lookup, rotatable, never logged in plaintext.

QR token and check-in

Token payload: { registrationId, eventId, userId, jti, issuedAt, expiresAt }. Sign server-side with CHECKIN_TOKEN_SECRET; target a 60-second lifetime, refresh near 25 seconds, and allow about 5 seconds of clock skew.

```
Authenticate scanner -> authorize exact event -> rate-limit -> verify signature/expiry/event -> BEGIN -> insert
jti redemption -> lock/validate confirmed registration -> insert attendance with unique(event_id,user_id) ->
write audit metadata -> COMMIT.
```

- A replay fails on the unique redemption jti even if the token is still within its expiry.
- Attendance is written with the privileged writer connection.
- Manual check-in requires organizer/admin permission and a reason; it is audited.
- Do not embed a bare registration ID or any signing secret in the client.

Concurrency and test ownership

Scenario	Required test
Last seat	20 parallel requests for one seat yield exactly one confirmed result; remaining requests waitlist/fail by policy.
Duplicate register	Parallel same-user requests leave one row and return ALREADY_REGISTERED.
Waitlist promotion	Concurrent cancellation/register preserves capacity and promotes the earliest valid row once.
Team final slot	Parallel joins never exceed max_team_size.
QR replay	Same jti used concurrently records one attendance row.
Cross-event scanner	Valid token plus wrong scanner scope is denied before write.

Completion criteria

- Unique constraints, lock order, transaction isolation, and retries are proven by integration tests against MySQL.
- No listing exposes unnecessary payment or personal data.
- Every organizer/scanner operation uses event-scoped authorization.
- Capacity, team size, waitlist, and check-in invariants hold under parallel test load.

7 Person 4 - Progression and platform

7. Person 4 - XP, achievements, payments, announcements, and storage

Mission

Own privileged progression and money-adjacent workflows, durable user uploads, announcements, and their audit/operational controls.

Owned repository operations

Area	Operations
progression	getSummary; listLedger; leaderboard; awardXp; recalculateTotal
achievements	list; listForUser; grant; markSeen
payments	listForUser; listForEvent; get; submitReceipt; verifyReceipt
announcements	list; create; archive
storage	createUploadUrl; completeUpload; createDownloadUrl; deleteObject

XP ledger and leaderboard

- xp_ledger is append-only. Each award has user, amount, reason, source type/source ID, actor, idempotency key, and created_at.
- awardXp uses the writer connection and a unique source/idempotency constraint so retries cannot double-award.
- Update characters.total_xp in the same transaction as the ledger insert, or derive it and use recalculateTotal as a controlled repair job.
- Never accept an arbitrary XP amount or actor ID directly from an untrusted client.
- Leaderboard reads the database view and applies deterministic ranking/tie-breaking.

Achievement grants

Evaluate eligibility from authoritative server data -> insert user_achievement using unique(user_id, achievement_id) -> write audit/source metadata -> optionally enqueue notification -> commit. markSeen is user-scoped and may update only the caller's record.

Payment receipt lifecycle

State / action	Rules
submitReceipt	Authenticated eligible participant; validate file metadata; storage object must be completed; one active receipt per registration/payment; status SUBMITTED.
review	Payment reviewer/admin only, preferably with 2FA assurance; read event-scoped minimum profile data.
verifyReceipt	Lock receipt and related registration; enforce valid transition; set VERIFIED or REJECTED with reviewer/time/reason; audit; trigger registration consequence atomically where required.
resubmission	Only after rejection and per policy; preserve prior receipt history rather than overwriting audit evidence.

Storage workflow

```
createUploadUrl -> validate purpose, type, size, owner, and object key -> return short-lived signed upload.
completeUpload -> HEAD/verify object metadata -> attach object to receipt/profile domain record idempotently.
createDownloadUrl -> authorize exact resource -> return short-lived signed download.
deleteObject -> require domain deletion/retention decision -> remove or quarantine; audit privileged deletion.
```

- Allow-list MIME type and extension, randomize object keys, strip user filenames from storage paths, and cap size.

- Use private buckets by default; never store public write credentials in NEXT_PUBLIC variables.
- Plan malware/content scanning before a receipt becomes reviewable.
- If storage fails, return STORAGE_UNAVAILABLE and preserve a retrievable application state.

Announcements

- list returns published, audience-eligible announcements in published_at/id order.
- create requires admin or event-specific announcement permission; validate audience, schedule, and safe content.
- archive is a reversible state transition, not destructive deletion; create an audit record.

Test ownership and completion criteria

- Idempotent XP, duplicate achievement, payment transition, storage completion, signed URL authorization, and announcement-scope tests pass.
- Normal application credentials cannot award XP, grant achievements, verify payments, or record privileged audit writes.
- Receipts remain private and download URLs are short-lived and authorization-scoped.
- Every privileged decision records actor, target, before/after state or reason, timestamp, and correlation ID.

8 File ownership and merge boundaries

8. File ownership and merge boundaries

Ownership is explicit to reduce conflicts. Ownership means primary implementation and required review, not exclusive knowledge.

Path / area	Primary	Required reviewer	Change rule
src/backend/db/**	P1	Affected domain owner	P1 allocates migrations and reviews all schema/connection changes.
drizzle.config.ts; scripts/check-schema.*	P1	Release owner	No bypass of schema checks.
src/backend/repositories/repository.ts	Shared / P1 custodian	All affected owners	ADR plus contract tests for breaking changes.
src/backend/repositories/errors.ts	Shared / P1 custodian	All affected owners	Error additions documented; removals are breaking.
src/backend/auth/**; middleware.ts	P2	Security reviewer	P2 required on identity/session changes.
src/backend/services/profiles/**; characters/**	P2	P1 for schema	P4 reviews progression-field interactions.
src/backend/services/events/**	P3	P2 for auth; P1 for schema	Cache and event-scope impacts documented.
registrations/**; teams/**; attendance/**	P3	P1 transaction review; P2 auth review	Concurrency test evidence required.
progression/**; achievements/**	P4	P1 grants/transaction review	Writer connection and idempotency evidence required.
payments/**; storage/**	P4	P2 auth; P1 schema	Security, privacy, and lifecycle review required.
announcements/**	P4	P3 event-scope review	Audience and event ownership tests required.
tests/contract/**	Shared	All owners	Must pass before any domain merge.

Migration ownership protocol

- Person 1 maintains the next migration number and a short migration ledger in the repository.
- A domain owner proposes schema changes in the same pull request as implementation or in a preceding schema-only pull request.
- Migrations are forward-only in shared environments. Each migration includes rollback/mitigation notes even when automatic down migration is unsafe.
- No developer edits an already-merged migration. Corrections are new migrations.
- Shared enum/status changes include compatibility code that allows mixed application versions during rollout.

Shared file protocol

- Create one short-lived coordination branch/PR for intentional contract updates; avoid drive-by formatting in shared files.
- Use generated schema/type artifacts where practical so merge conflicts are detected immediately.
- A pull request touching more than one lane lists each owner and identifies the exact cross-lane invariant affected.

9

Branch and pull request strategy

9. Branch and pull request strategy

Branch model

Use trunk-based development with short-lived feature branches. main remains releasable; preproduction is an environment, not a long-lived divergent code branch.

Branch pattern	Use	Maximum lifetime
feat/p1-db-	Foundation/schema work	2-3 working days
feat/p2-auth-	Identity/access work	2-3 working days
feat/p3-events-	Event operations	2-3 working days
feat/p4-platform-	Progression/platform work	2-3 working days
fix/-	Focused defect	1-2 working days
release/	Only when a stabilization branch is necessary	Short freeze window

Commit and pull request rules

- One logical change per commit; include migrations, code, tests, and documentation needed to understand that change.
- Rebase/update from main before final review; never rewrite main.
- Pull requests target main and use squash merge unless preserving a deliberate migration sequence is important.
- Required checks: formatting, lint, typecheck, unit, contract, MySQL integration, migration-from-zero, schema drift, security/static scan, and build.
- Required approvals: primary code owner plus cross-owner for schema, authorization, writer, or shared-contract changes.
- No secrets, personal receipt data, access tokens, join codes, or production dumps in commits, fixtures, logs, or screenshots.

Pull request template

```
Summary and owner
Source requirement / operation(s)
Interface or schema changes
Transaction and lock order
Authorization and data exposure
Migration / rollback notes
Tests and concurrency evidence
Observability / runbook impact
Screenshots or API examples when useful
```

Definition of done for a pull request

Gate	Question
Contract	Does the implementation match the typed interface and stable DataError behavior?
Security	Are actor identity, authorization, validation, rate limit, secrets, and data minimization correct?
Data	Are constraints, indexes, transaction boundaries, lock order, and migration compatibility correct?
Tests	Do unit, integration, contract, authorization, and concurrency tests cover the change?
Operations	Are logs, metrics, alert implications, idempotency, and recovery documented?
Review	Have all named owners approved and all discussions been resolved?

10 Transaction and consistency catalog

10. Transaction and consistency catalog

This catalog is the merge-time source of truth for multi-row mutations. Lock resources in the listed order and keep external network calls outside database locks wherever possible.

Flow	DB role	Lock order	Atomic writes	Idempotency / constraint
Sign-up	app	email/user lookup -> new user	user, profile, default role or pending role assignment	UNIQUE normalized email; token uniqueness
Register	app (+ progression seam)	event -> registrations	registration status and source event	UNIQUE(event_id,user_id)
Cancel/promote	app (+ progression seam)	event -> caller registration -> earliest waitlist	cancel plus one promotion	deterministic waitlist ordering
Create team	app	event -> new team	team plus leader membership	UNIQUE join_code
Join team	app	team -> memberships	membership insert	UNIQUE(team_id,user_id); size lock
Leave team	app	team -> memberships -> registration	remove, transfer/dissolve, registration validity	leader rule and deadline
Check-in	writer	jti redemption -> registration -> attendance	redemption, attendance, audit	UNIQUE jti; UNIQUE(event_id,user_id)
Award XP	writer	character -> source/idempotency row	ledger insert plus total update	UNIQUE(source_type,source_id, user_id) or idempotency_key
Grant achievement	writer	user achievement key	grant plus source/audit	UNIQUE(user_id,achievement_id)
Verify payment	writer	receipt -> registration	status/reviewer/reason plus registration consequence and audit	valid state transition
Complete upload	app	object reference/domain row	attach verified object metadata	UNIQUE object key; completion idempotency

Deadlock and retry policy

- Use a common lock order and small transactions first; retry is a fallback, not a substitute.
- Retry only deadlock 1213 and lock wait timeout 1205, maximum three total attempts.
- Use 50 ms then 100 ms backoff with jitter if practical.
- The entire transaction callback must be safe to rerun; external email/storage calls are not performed inside the retried callback.
- Log retry count, operation, correlation ID, and anonymized resource identifiers.

Outbox / post-commit side effects

Email, notification, certificate creation, and cross-domain progression actions should use a transactional outbox or durable job record when their failure would otherwise leave the database committed but the side effect lost. A worker retries with an idempotency key.

Invariants to assert

- confirmed registrations <= event capacity
- team member count <= event max team size
- at most one active registration per user/event
- at most one attendance per user/event and one token redemption per jti
- character total XP equals the accepted ledger total or is repairable by a controlled reconciliation

- payment status transitions follow the defined state machine
- roles and audit history cannot be written by the normal application role

Reconciliation and consistency controls

Scheduled reconciliation jobs are a safety net for detection and repair, not permission to weaken transactional guarantees. Each job is read-only by default and produces a reviewable exception report.

Check	Detection query / signal	Repair authority
Event capacity	confirmed count above capacity or negative seats_remaining	P3 investigates; project lead approves participant-impacting correction.
Waitlist order	Promoted row is not earliest eligible waitlisted record	P3 replays from audit history using controlled tool.
Team size	membership count outside event min/max policy	P3 and event owner resolve before registration/check-in.
Attendance	attendance without confirmed registration or duplicate semantic check-in	P3; manual correction requires reason and audit.
XP total	characters.total_xp differs from accepted ledger sum	P4 recalculateTotal through writer connection; preserve ledger.
Payment state	verified receipt inconsistent with registration requirement/state	P4 reviewer plus P3 domain owner.

Consistency incident response

- Disable the affected mutation with a feature flag when an invariant can continue to drift.
- Preserve logs, audit rows, correlation IDs, and a database snapshot before repair.
- Quantify affected records with a read-only query and identify the earliest bad release/migration.
- Use an approved, idempotent repair command with dry-run output; never edit production rows manually.
- Add the incident scenario to the regression and concurrency suite before re-enabling the flow.

11 Security and privacy rules

11. Security and privacy rules

Identity and session

- Passwords are hashed server-side with a modern adaptive algorithm; never log or transport them outside TLS.
- Cookies are httpOnly, Secure, SameSite=Lax/Strict, scoped, rotated after authentication, and revoked on password reset.
- Email/reset/check-in tokens are purpose-scoped, signed or hashed, short-lived, one-time where applicable, and validated against server time.
- Return generic credential/reset responses to reduce account enumeration.
- Require 2FA for admin and high-impact operational roles.

Authorization and data access

- Authentication answers who; authorization answers whether that actor may perform this exact action on this exact resource.
- Mutations reread role and scope from the database. Middleware and UI visibility are defense-in-depth only.
- Use NOT_FOUND when revealing resource existence would leak sensitive information.
- Return minimum DTOs: public event data, self profile data, and event-scoped operational data are different shapes.
- MySQL lacks application row-level security; service authorization and least database grants are mandatory.

Input, abuse, and content controls

Surface	Controls
Auth/reset	Zod validation; normalization; rate limits by account/network; bot monitoring; generic errors.
Join code	Entropy; rate limit; constant-shape failure; rotation; no logs/analytics capture.
QR check-in	HMAC; expiry; jti replay table; scanner/event scope; rate limit; audit.
Receipt upload	Purpose-bound signed URL; type/size allow-list; object key randomization; private bucket; malware scan.
Announcements/profile text	Length/schema checks; safe rendering; no trusted HTML from users.
Admin mutations	2FA assurance; CSRF/session controls; idempotency; audit; tighter rate limit.

Secrets and environment

- Commit only .env.example; production/preproduction secrets live in the deployment secret manager.
- AUTH_SECRET, CHECKIN_TOKEN_SECRET, database URLs, storage credentials, OAuth secrets, and SMTP credentials are server-only.
- Rotate any credential ever pasted into chat, documents, logs, or issues.
- Use separate credentials and buckets per environment; deny production access from developer machines except through approved operational access.

Audit logging

Audit privileged role grants, payment decisions, attendance/manual check-in, XP/achievement grants, security-setting changes, storage deletion, and announcement administration. Store actor, action, target type/ID, time, result, reason, correlation ID, and safe before/after metadata. Do not store secrets or full receipt content in logs.

Retention and privacy

- Define retention for sessions, verification/reset tokens, payment receipts, audit logs, attendance, and certificates.
- Support account/privacy requests without destroying legally or operationally required audit evidence.
- Use redaction or anonymization for test data and support bundles.
- Backups and exports inherit the same access and retention classification as source data.

Required security evidence

Control	Evidence before release
Secret isolation	Client-bundle scan and repository history scan show no server secret or credential.
Database privilege	Automated grant test proves app-role denial and writer-role minimum access.
Authorization	Role/resource matrix includes wrong-role and right-role/wrong-resource negative tests.
Session/token	Expiry, revocation, replay, clock-skew, rotation, and generic-error tests pass.
Upload privacy	Private bucket, signed URL expiry, MIME/size enforcement, and unauthorized download denial verified.
Audit	Sample role, payment, attendance, XP, and announcement actions are queryable by correlation ID.
Dependencies	Dependency and static scans have no unresolved critical/high finding or documented exception.

Security incident readiness

- Document owners and rotation steps for database, Auth, QR, OAuth, SMTP, and storage credentials.
- Maintain a feature-flag path to stop registration, check-in, payment review, and privileged award mutations independently.
- Prepare queries for recent role grants, payment decisions, token replays, unusual authorization denials, and storage downloads.
- Preserve audit evidence and notify the project lead before any user-visible remediation or credential-wide logout.

12 Test ownership and quality strategy

12. Test ownership and quality strategy

Each lane owns tests for its methods; shared quality gates prevent one lane from declaring completion while breaking another.

Suite	P1	P2	P3	P4	Merge gate
Unit	DB helpers, mapping, config	auth/authz/profile	eligibility and state rules	XP/payment/storage rules	All changed modules
Contract	harness and shared types	auth/profile/character	event/registration/team/attendance	progression/payment/announcement/storage	All 58 operations
MySQL integration	migrations, views, grants	auth adapter and roles	locks and constraints	writer flows and audit	Required
Authorization	DB grants	role/scope matrix	organizer/scanner scope	reviewer/admin/storage scope	Required
Concurrency	retry harness	role grant idempotency	capacity/team/check-in	XP/achievement/payment idempotency	Required for mutations
E2E	environment fixture	sign-up/sign-in/reset	register/team/check-in	receipt/verify/XP/announcement	Critical journeys
Performance	pool and query plans	auth endpoints	event/list/register spikes	leaderboard/receipt queues	Pre-release

Minimum test layers

- **Unit:** pure validation, policy, mapping, token, and state-transition logic.
- **Repository integration:** real MySQL for constraints, isolation, locking, views, and grants.
- **Contract:** every repository implementation returns the same shape and `DataError` semantics expected by callers.
- **Authorization:** anonymous/player/organizer/scanner/reviewer/admin and cross-resource negative cases.
- **Concurrency:** parallel attempts at the last seat, final team slot, same check-in token, same XP source, and same payment decision.
- **E2E:** one complete participant journey and one organizer/reviewer journey in preproduction.

Required fixtures

- Anonymous, unverified user, verified player, two organizers assigned to different events, scanner, payment reviewer, and admin.
- Events in draft/open/closed/full/waitlist states; individual and team modes; free and payment-required variants.
- Teams at min/max size, leader/member cases, expired/rotated join code.
- Submitted/verified/rejected receipt states and private storage objects.
- XP and achievement sources with duplicate/idempotency cases.

Coverage and evidence

Prioritize invariant and decision coverage over a single percentage. As a baseline, changed backend code should maintain at least 80% line coverage, but merge approval also requires explicit negative authorization, transaction rollback, constraint, and concurrency evidence.

Performance targets to agree before load test

Path	Initial target	Load profile
Public event list/detail	p95 <= 300 ms excluding network/CDN	Read-heavy launch traffic; cache warm/cold.
Registration	p95 <= 800 ms under expected burst	Concurrent requests around capacity boundary.
QR check-in	p95 <= 500 ms	Venue burst with replay and wrong-scope noise.
Leaderboard	p95 <= 400 ms	Read-heavy; view/index validation.
Receipt review list	p95 <= 500 ms	Organizer/reviewer pagination and private download authorization.

Load-test design

- Model launch bursts, not only steady averages: event-list fan-out, last-seat contention, venue check-in waves, and receipt-review batches.
- Use production-like indexes and data volume in preproduction; warm and cold cache runs are separate.
- Capture database query plans, lock waits, retries, pool saturation, CPU/memory, queue backlog, and error distribution with latency.
- Verify correctness after load by running every invariant/reconciliation query, not just checking response success.

CI gate order

Format/lint -> typecheck -> unit -> contract -> migration-from-zero -> MySQL integration -> authorization -> concurrency -> build -> security scans -> preview smoke. Fast failures run first; no later green job overrides an earlier failed invariant or security gate.

Test-data lifecycle

- Generate deterministic synthetic fixtures; never copy production users or receipts into tests.
- Use a unique run ID for parallel CI databases and object prefixes.
- Clean up through scoped test teardown or disposable databases/buckets; never target a shared root path.
- Keep failing test artifacts limited to redacted IDs, safe logs, query plans, and screenshots without personal data.

13 Integration sequence and release train

13. Integration sequence and release train



Wave 0 - contract freeze

- Approve repository method names, DTOs, DataError codes, ID/date conventions, and authorization helper signatures.
- Create CODEOWNERS, PR template, CI skeleton, architecture decision records, and migration ledger.
- Agree event, registration, payment, receipt, and achievement status machines.

Wave 1 - database foundation

- Merge config validation, connections, core schema, migrations, seed, shared errors, transactions, retries, and grant tests.
- Publish stable schema/types so lanes can implement in parallel.

Wave 2 - identity plus read paths

- Merge Auth.js/session/authorization/profile foundations.
- Merge reference, public events, schedule, stats, leaderboard, announcements, and user-scoped read APIs.
- Frontend integrates against real read contracts while mutations remain feature-flagged.

Wave 3 - core participant transactions

- Merge registration/cancellation, team create/join/leave, and their concurrency suites.
- Integrate profile verification, payment requirement hooks, and progression seams without direct cross-repository SQL.

Wave 4 - privileged operational flows

- Merge QR check-in, XP/achievement grants, payment verification, signed storage, role grants, and full audit controls.
- Prove writer-role isolation and idempotency under concurrent retries.

Wave 5 - hardening and release

- Run full E2E, performance, security, backup/restore, migration rehearsal, and operational drills in preproduction.
- Freeze schema changes, resolve P0/P1 issues, publish runbooks, complete go-live gates, and deploy with staged enablement.

Integration ownership

Integration seam	Driver	Participants	Evidence
Schema and repositories	P1	All	Migration CI and 58-operation contract suite.
Auth in every mutation	P2	P3/P4	Authorization matrix and cross-event negatives.
Registration -> progression/payment	P3	P4/P2	Idempotent integration events and E2E journey.
Check-in -> XP/achievement	P3	P4	Atomic attendance plus idempotent progression side effect.
Payment -> registration	P4	P3/P2	Valid transition, authorization, audit, E2E.
Release	Named release owner	All	Signed gate checklist and rollback drill.

Wave exit criteria

Wave	Exit criteria
0	Repository types compile; DataError catalog and state machines approved; CODEOWNERS and CI active.
1	Fresh MySQL migration/seed succeeds; both connections verified; protected-write denial proven.
2	Authentication and all read contracts pass; public/self/organizer DTO boundaries verified.
3	Capacity, waitlist, team-size, and rollback concurrency tests pass repeatedly against MySQL.
4	QR replay, XP/achievement idempotency, payment transitions, private storage, and audit tests pass.
5	All go-live gates pass; UAT signed; backup/restore and rollback drills completed.

Cross-lane integration smoke suite

- New user verifies identity, creates a character, and reads eligible events.
- Verified player registers; correct confirmed/waitlist and payment state appears in self and organizer views.
- Team leader creates a team, member joins, and size/registration rules stay consistent.
- Receipt upload completes privately; authorized reviewer verifies; registration consequence is applied once.
- Authorized scanner checks in; replay is denied; attendance and eligible XP/achievement appear once.
- Announcement is visible only to its intended audience and archived state disappears from active reads.

14 Go-live gates and operations

14. Go-live gates and operations

Release requires evidence, not verbal confidence. A gate is complete only when the named artifact, test run, or operational drill exists.

Gate	Pass condition	Owner	Evidence
G1 Scope/contract	All 58 methods mapped; DTO/error/version baseline signed.	All	Contract inventory and passing suite.
G2 Schema	Fresh migration, upgrade rehearsal, drift/index/constraint check pass.	P1	CI and preproduction migration report.
G3 Least privilege	App role denied protected tables; writer secret server-only.	P1/P4	Grant test and secret scan.
G4 Identity	Session, reset, OAuth, verification, 2FA, role/scope tests pass.	P2	Security test report.
G5 Concurrency	Capacity, team, QR, XP, payment parallel tests preserve invariants.	P3/P4	Repeatable MySQL test output.
G6 Storage/privacy	Private objects, signed URLs, scan, retention, redaction verified.	P4	Storage/privacy checklist.
G7 Performance	Agreed p95/error targets met at expected peak plus headroom.	All	Load-test report and query plans.
G8 Observability	Dashboards, logs, traces/correlation IDs, alerts, and audit queries work.	Release owner	Alert test and dashboard links.
G9 Recovery	Backup restore, rollback/forward-fix, secret rotation, and queue replay drilled.	P1/Release	Timed drill record.
G10 E2E/UAT	Participant and organizer journeys pass in preproduction.	All	Signed UAT checklist.
G11 Defects	No open P0/P1; accepted P2s have owner and mitigation.	Release owner	Release issue list.
G12 Launch approval	Runbooks, on-call coverage, comms, flags, and rollback authority assigned.	Project lead	Go/no-go record.

Deployment sequence

- Announce change window and verify current backup/restore point.
- Apply backward-compatible migrations and validate schema/version/grants.
- Deploy application with high-risk mutations behind flags.
- Run smoke tests: sign-in, event read, registration fixture, storage signing, authorized check-in, audit query.
- Enable participant flows gradually; watch database locks, errors, latency, pool saturation, and queue backlog.
- Enable privileged organizer/payment/check-in features only after participant metrics are stable.
- Record release version and migration version in the operational log.

Rollback / forward-fix triggers

- Confirmed registrations exceed capacity or duplicate attendance/XP/payment decisions appear.
- Authorization allows cross-event or cross-user access.
- Application credential can write protected tables.
- Migration causes data loss, sustained lock contention, or incompatible application behavior.
- Error rate, latency, pool saturation, or storage failures exceed agreed launch thresholds.

Operational dashboards

Dashboard	Signals
API health	request rate, p50/p95/p99, 4xx/5xx by operation, correlation IDs
Database	pool usage, slow queries, deadlocks, lock waits, transaction retries, replica/backup state
Registration	confirmed/waitlisted/cancelled, capacity invariant, promotion failures
Check-in	success/denied/replay/expired/wrong-scope, venue/event distribution
Auth/security	login failure, reset/verification rate, rate-limit events, role grants, suspicious scope denials
Platform	XP awards/duplicates, receipt states, storage failures, queue retries, announcement publish failures

Launch-day command roles

Role	Authority and responsibility
Release commander	Runs checklist, controls deploy/flags, makes go/no-go and rollback call.
Database owner (P1)	Monitors migrations, pool, locks, grants, backups, and database rollback/forward-fix.
Security/identity owner (P2)	Monitors auth, scopes, role changes, token/session anomalies, and identity comms.
Operations owner (P3)	Monitors event capacity, waitlist, team, registration, and venue check-in health.
Platform owner (P4)	Monitors XP, achievements, payments, storage, announcements, and background jobs.
Communications owner	Maintains stakeholder/user status messages approved by release commander.

Incident severity during launch

- **SEV-0:** security breach, destructive data loss, or systemic unauthorized access - stop affected services and escalate immediately.
- **SEV-1:** invariant breach, payments/check-ins materially wrong, or broad outage - disable affected mutation and decide rollback.
- **SEV-2:** degraded performance or partial non-critical failure with workaround - assign owner, mitigate, and monitor.
- **SEV-3:** cosmetic or low-impact defect - record for normal triage without disrupting launch.

15 Team operating model

15. Team operating model

RACI summary

Work item	P1	P2	P3	P4	Lead/release
Schema/migrations	A/R	C	C	C	I
Repository contract	C	C	C	C	A
Authentication/authorization	C	A/R	C	C	I
Events/registration/team/attendance	C	C	A/R	C	I
XP/achievement/payment/storage	C	C	C	A/R	I
Security review	C	R	C	C	A
Integration test	R	R	R	R	A
Release/go-no-go	C	C	C	C	A/R

R Responsible; A Accountable; C Consulted; I Informed.

Cadence

- **Daily 15 minutes:** interface/schema blockers, cross-lane dependencies, risky merges, and preproduction health.
- **Twice-weekly integration window:** merge shared contract/schema changes with all affected owners available.
- **Weekly risk review:** concurrency, security, privacy, migrations, performance, and launch readiness.
- **Before each wave:** entry criteria review; after each wave: evidence review and retrospective.

Decision records required

- Session strategy and Auth.js adapter decisions.
- Exact payment and registration state machines.
- XP source/idempotency model and reconciliation approach.
- Team leader-leave and below-minimum-size policy.
- QR signing algorithm, rotation, and offline/clock-skew policy.
- Object storage provider, malware scanning, retention, and deletion policy.
- Outbox/job infrastructure and delivery guarantees.

Escalation rules

- Stop merging if a shared interface is ambiguous; resolve it in an ADR first.
- A security or invariant failure blocks the owning wave and release.
- Schema conflicts are resolved by Person 1 with affected owners; no one bypasses migration history.
- A product-policy ambiguity that changes money, eligibility, XP, team state, or retention is escalated to the project lead.

A Appendix A - Operation acceptance matrix

Appendix A - Operation acceptance matrix

Every method is complete only when authorization, validation, output shape, expected `DataError`, tests, and observability are present.

Area	Operation	Actor / authorization	Key acceptance condition
auth	signUp	anonymous	Normalized unique email; server hash; atomic identity bootstrap; verification flow.
auth	signIn	anonymous	Generic <code>INVALID_CREDENTIALS</code> ; secure session cookie; rate limit.
auth	signInWithProvider	anonymous	State/nonce validation; safe account link; no provider role trust.
auth	signOut	authenticated	Session revoked and cookie cleared.
auth	getSession	any	Safe session DTO; no secrets; expiry honored.
auth	onAuthStateChange	client integration	Adapter-compatible state change without trusting client mutation.
auth	grantRole	admin + 2FA	Writer role; DB reread; audit; idempotent grant.
profiles	get	self/public/event-scoped	Field-minimized DTO by caller context.
profiles	update	self	Allow-list only; protected fields rejected; audit sensitive change.
reference	listColleges	public	Stable seeded order.
reference	listDepartments	public	Optional college filter; stable seeded order.
reference	listCategories	public	Seven seeded categories.
reference	listLevels	public	Seven seeded levels; thresholds ordered.
reference	listSponsors	public	Only active/public sponsor fields.
characters	getForUser	self/public DTO	No protected progression mutation fields.
characters	create	verified self	Unique player name; default progression values.
characters	update	self	<code>PLAYER_NAME_TAKEN</code> handling; progression fields immutable.
events	list	public	Filters, pagination/cache, safe public DTO.
events	getBySlug	public/session-aware	Stable not-found and user-specific state when authenticated.
events	getById	public/session-aware	Same domain contract as slug lookup.
events	stats	public summary / organizer detail	<code>event_stats</code> view; scoped detailed data.
events	schedule	public	Deterministic <code>starts_at</code> ordering.
registrations	listForUser	self	User event/status/payment/team/check-in summary.
registrations	listForEvent	event organizer/admin	Minimum participant data; pagination.
registrations	get	self or event organizer	Ownership/scope; safe <code>NOT_FOUND</code> .
registrations	register	verified eligible user	Event lock; capacity invariant; unique registration.
registrations	cancel	self / authorized operator	Event-first lock; deterministic promotion.
teams	listForUser	self	Only memberships visible to caller.
teams	getById	member/authorized event operator	Safe team DTO and scope.
teams	getByJoinCode	authenticated + rate limited	Constant-shape <code>INVALID_JOIN_CODE</code> ; no leakage.
teams	members	member/event organizer	Field-minimized member list.
teams	create	eligible verified user	Team plus leader atomic; unique random code.
teams	join	eligible verified user	Team lock; max size; membership uniqueness.
teams	leave	member	Leadership/deadline/min-size policy enforced atomically.
attendance	listForUser	self	Own attendance only.

Area	Operation	Actor / authorization	Key acceptance condition
attendance	listForEvent	event organizer/scanner scope	Operational minimum fields.
attendance	checkIn	authorized scanner	Signed token; jti replay defense; unique attendance; audit.
progression	getSummary	self/public projection	Total/level derived consistently.
progression	listLedger	self/admin scope	Cursor pagination; safe reasons/sources.
progression	leaderboard	public	View-backed deterministic rank.
progression	awardXp	trusted service/admin	Writer connection; source idempotency; atomic total.
progression	recalculateTotal	admin/job	Audited repair; matches accepted ledger.
achievements	list	public	Stable achievement definitions.
achievements	listForUser	self/public projection	Seen state private unless allowed.
achievements	grant	trusted service/admin	Writer connection; unique grant; audit.
achievements	markSeen	self	Caller-owned row only.
payments	listForUser	self	Own receipt/payment status only.
payments	listForEvent	reviewer + event scope	Private pagination; minimum personal fields.
payments	get	owner/reviewer scope	Private signed download only after authorization.
payments	submitReceipt	eligible self	Completed private object; active receipt uniqueness.
payments	verifyReceipt	reviewer/admin + 2FA	Locked valid transition; reviewer/reason/audit.
announcements	list	audience-eligible	Published/active only, stable order.
announcements	create	admin/event manager	Audience/schedule/content validation; audit.
announcements	archive	admin/event manager	Reversible state change; audit.
storage	createUploadUrl	authorized owner	Purpose/type/size-bound short-lived private upload.
storage	completeUpload	authorized owner	HEAD metadata check; idempotent domain attachment.
storage	createDownloadUrl	resource-authorized caller	Short-lived URL; no public bucket exposure.
storage	deleteObject	owner by policy/admin	Retention checked; audited delete/quarantine.

B

Appendix B - Schema ownership matrix

Appendix B - Schema ownership matrix

Table	Behavior owner	Critical constraints / notes
users	P2	CHAR(36) ASCII ID; unique normalized email; account status.
accounts	P2	Auth.js provider/account uniqueness; secrets/tokens protected.
sessions	P2	Expiry and revocation indexes; private token material.
verification_tokens	P2	Unique hashed token; purpose and expiry.
profiles	P2	One-to-one user; protected verification/role fields absent.
user_roles	P2	Unique user/role/scope; writer-only mutation; audit.
colleges	P1	Stable seed key/name; active flag.
departments	P1	College relation if applicable; stable seed key.
characters	P2/P4	Unique player name; nonnegative total_xp; stable created_at.
levels	P4/P1	Ordered thresholds; deterministic seed.
xp_ledger	P4	Append-only; idempotent source; writer-only.
achievements	P4	Stable key; rule metadata; deterministic seed.
user_achievements	P4	Unique user/achievement; grant and seen timestamps.
event_categories	P1/P3	Seven seeded categories; stable slug.
events	P3	Unique slug; capacity/window/team/payment constraints.
event_organizers	P2/P3	Unique event/user assignment; scope source of truth.
schedule_slots	P3	Event FK; starts/ends validation; order index.
announcements	P4/P3	Audience/event scope; publish/archive timestamps.
teams	P3	Event FK; unique join code; leader policy.
team_members	P3	Unique team/user; role; joined_at; size protected by lock.
registrations	P3	Unique event/user; status/payment/team links; ordered waitlist.
attendance	P3	Unique event/user; writer-only; scanner/method metadata.
checkin_token_redemptions	P3	Unique jti; token/event/user metadata; retention.
payment_receipts	P4	Object reference; state machine; reviewer/reason/timestamps.
audit_log	Shared/P4 infrastructure	Append-only privileged audit; safe structured metadata.
sponsors	P1/P4	Stable seed/public fields; active flag.
certificates	P4	Unique source/user certificate; object/verification metadata.

C

Appendix C - Environment contract

Appendix C - Environment contract

Exact provider-specific names may vary, but every environment must expose a validated equivalent.

Variable	Scope	Purpose / rule
DATABASE_URL	server	Least-privilege application MySQL connection; TLS required.
WRITER_DATABASE_URL	server privileged	Sensitive writes only; never imported into general/client modules.
AUTH_SECRET	server	Auth/session signing or encryption; rotate through managed procedure.
CHECKIN_TOKEN_SECRET	server	HMAC key for short-lived QR tokens; independent from AUTH_SECRET.
STORAGE_BUCKET_URL / BUCKET	server	Private object-storage target per environment.
STORAGE_ACCESS_KEY	server	Storage access identity; minimum bucket permissions.
STORAGE_SECRET_KEY	server secret	Never logged or exposed to client.
SMTP_URL	server secret	Verification/reset mail transport; separate environment credentials.
OAuth provider IDs/secrets	server	Exact redirect URIs and environment separation.
APP_BASE_URL	server/public config	Canonical URL used for callbacks and absolute links.
LOG_LEVEL / RELEASE	server	Operational metadata; no sensitive values.

Environment promotion checklist

- Preproduction and production schema versions are known and reproducible.
- Secrets are unique by environment and absent from repository history/client bundles.
- OAuth/SMTP/storage callbacks and permissions are tested in the target environment.
- Application and writer database grants match the approved matrix.
- Clock synchronization, UTC behavior, backup schedule, and alert destinations are verified.

D

Appendix D - Review checklists

Appendix D - Review checklists

Transaction review

- What invariant is protected?
- What rows are locked, in what order, and why?
- Can the callback safely retry after deadlock?
- Which unique/check constraints provide the final defense?
- Are external calls outside locks or represented by an outbox?
- Does rollback remove every partial state?

Authorization review

- Is actor identity taken only from the server session?
- Is the role/scope reread from the database for mutation?
- Does event/resource scope match the exact target?
- Is the returned DTO the minimum required?
- Do negative tests cover anonymous, wrong role, and correct role/wrong resource?

Migration review

- Is the migration forward-compatible with the currently deployed application?
- Are foreign-key types/charsets identical and indexes named?
- Is data backfill bounded, resumable, and safe for existing rows?
- Are grants, views, triggers (if any), and schema checks updated?
- Has migrate-from-zero and upgrade-from-current been tested?
- Is rollback or forward-fix documented?

Release review

- Are all gates in Section 14 evidenced and signed?
- Are feature flags, owners, dashboards, alerts, and on-call contacts assigned?
- Has backup restore and rollback/forward-fix been rehearsed?
- Are P0/P1 issues zero and remaining risks explicitly accepted?
- Does the release log record code version, migration version, time, and decision owner?

End of plan. This document should be versioned with the repository and updated through reviewed changes whenever the shared contract, schema, ownership, or release policy changes.